Persistência de Dados com SQLite no Flutter

Prof Dr Alencar Machado

https://www.sqlite.org/docs.html

https://docs.flutter.dev/cookbook/persistence/sqlite

O que é SQLite?

- Banco de dados relacional embutido
- Zero-configuration
- Auto-contido
- Ideal para dispositivos móveis
- **Vantagens:**
  - Leve (≈250KB)
  - ACID-compliant
  - Suporte multiplataforma

SQLite

Características:

- ❌ Sem suporte a acesso concorrente de escrita (apenas 1 escritor por vez)
- ❌ Tamanho máximo de 140 TB (teórico), mas desempenho cai após ~100 GB
- ❌ Sem sistema de usuários/roles nativo
- ❌ Tipagem fraca: INTEGER aceita strings
- ❌ Sem client-server: Dificulta acesso remoto
- ❌ Máx. 2000 colunas por tabela
- ✅ SQLite **tem triggers**
- ❌SQLite **não tem stored procedures** (igual a PostgreSQL, MySQL, etc.)
- Para lógicas complexas no banco, você depende apenas de **triggers** ou **aplicação externa**.

✅ Casos Ideais:

- Aplicativos mobile/desktop offline
- Prototipagem rápida
- Cache local de dados
- Dispositivos IoT com recursos limitados
- Testes unitários

⛔ Evitar Quando:

- 100 usuários simultâneos
- Dados críticos financeiros
- Necessidade de replicação multi-servidor

Foreign Keys com SQLite

Necessidade de ativação manual com: ***PRAGMA foreign_keys = ON;***

Sem Verificação em ALTER TABLE:

- Alterar esquemas pode quebrar relações sem aviso.

Restrições em DROP TABLE:

- Tabelas dependentes não são automaticamente removidas.

```
Future<Database> abrirBanco() async {
  final db = await openDatabase(...);
  await db.execute('PRAGMA foreign_keys = ON');
  return db;
}
```

SQLite

Feature	SQLite	PostgreSQL	MySQL	Firebase
Escalabilidade	Baixa	Alta	Média-Alta	Alta
Concorrência	Single-writer	Multi-writer	Multi-writer	Multi-writer
Armazenamento	Arquivo único	Servidor	Servidor	Cloud
Uso Típico	Mobile/IoT	Web Apps	Web Apps	Apps em tempo real

Configuração Inicial

1. Adicione as dependências: Necessário para usar o sqlite

```
dependencies:
  flutter:
    sdk: flutter
  sqflite: ^2.3.0
  path: ^1.9.0
```

2. Inicialização:

Na main.dart inicializar o banco, conforme configuramos a seguir.

```
Run | Debug | Profile
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await AppDatabase().database;

  runApp(const MyApp());
}
```

Configuração do Banco

Criar o arquivo .dart de para configurar as tabelas iniciais do banco.

Com isso, ao iniciar o app o banco criará as tabelas necessárias.

Passos:

- Defina o caminho do banco
- Crie tabelas no `onCreate`
- Controle versões

```
class AppDatabase {
  static final AppDatabase _instance = AppDatabase._internal();
  static Database? _database;

  factory AppDatabase() => _instance;

  AppDatabase._internal();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDatabase();
    return _database!;
  }

  Future<Database> _initDatabase() async {
    final path = join(await getDatabasesPath(), 'student_database.db');
    return openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE users(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT,
            email TEXT,
            password TEXT
          )
        ''');
        await db.execute('''
          CREATE TABLE students(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
```

Configuração do Banco - Comandos

onUpgrade

Quando é chamado:

- Quando o `version` do banco é **aumentado**

Propósito:

- Migrações de schema
- Atualizar estrutura sem perda de dados

```
onUpgrade: (db, oldVersion, newVersion) async {
  if (oldVersion < 2) {
    await db.execute('ALTER TABLE users ADD COLUMN birth_date TEXT');
  }
  if (oldVersion < 3) {
    await db.execute('CREATE TABLE settings(user_id INTEGER PRIMARY KEY, theme TEXT)');
  }
}
```

Model Layer

- **Definição:**
  Camada que representa a **estrutura de dados** e as **regras de domínio** da aplicação.
  "Fonte única da verdade" sobre como os dados são organizados.
- **Componentes:**
  - Classes Dart imutáveis
  - Métodos de conversão (Map ↔ Objeto)
  - Validações básicas
- **Responsabilidades:**
  - Representação dos dados
  - Serialização e deserialização

```dart
class User {
  final int? id;
  final String name;
  final String email;
  final String password;

  User({
    this.id,
    required this.name,
    required this.email,
    required this.password,
  });

  Map<String, dynamic> toMap() {
    return {
      'id': id,
      'name': name,
      'email': email,
      'password': password,
    };
  }

  factory User.fromMap(Map<String, dynamic> map) {
    return User(
      id: map['id'],
      name: map['name'],
      email: map['email'],
      password: map['password'],
    );
  }
}
```

Camada DAO (Data Access Object)

O que é o DAO?

- **Definição:**
  Padrão de projeto que **abstrai e encapsula** o acesso aos dados. Ponte entre o código e o banco de dados.
- **Analogia:**
  "Garçom" que entrega/pega dados do banco para a aplicação.

Responsabilidades Principais

CRUD (Create, Retrieve, Update, Delete)

```
class UserDao {
  static const String table = 'users';

  Future<int> insertUser(User user) async {
    final db = await AppDatabase().database;
    return db.insert(table, user.toMap());
  }

  Future<User?> getUser(String email, String password) async {
    final db = await AppDatabase().database;
    final result = await db.query(
      table,
      where: 'email = ? AND password = ?',
      whereArgs: [email, password],
    );
    return result.isNotEmpty ? User.fromMap(result.first) : null;
  }
}
```

Gerenciamento de conflito

ConflictAlgorithm.abort (Padrão)

Comportamento:

- Cancela a operação atual
- Mantém as alterações anteriores
- Retorna erro `SQLITE_CONSTRAINT`

```
await db.insert(
  'users',
  user.toMap(),
  conflictAlgorithm: ConflictAlgorithm.abort,
);
```

Cenário ideal: Quando queremos tratamento explícito de erros por código (ex: mostrar mensagem ao usuário).

Gerenciamento de conflito

ConflictAlgorithm.fail

Comportamento:

- Aborta o comando atual
- **Não reverte** transações anteriores
- Retorna erro `SQLITE_CONSTRAINT`

Diferença para abort:

Enquanto `abort` reverte mudanças do comando atual, `fail` deixa o banco no estado em que estava no início do comando.

Exemplo:

Útil em transações onde comandos anteriores devem ser mantidos mesmo após falha.

Gerenciamento de conflito

ConflictAlgorithm.ignore

Comportamento:

- Ignora silenciosamente o conflito
- Não insere/atualiza o registro
- **Não** gera erro

```
await db.insert(
  'users',
  user.toMap(),
  conflictAlgorithm: ConflictAlgorithm.ignore,
);
```

Caso de uso:

Quando tentativas duplicadas são aceitáveis e não requerem ação (ex: logs não críticos).

Gerenciamento de conflito

ConflictAlgorithm.replace

Comportamento:

- Substitui o registro existente
- Mantém a restrição de chave primária
- Atualiza outros campos

```
// Se email é UNIQUE:
await db.insert(
  'users',
  {'email': 'a@a.com', 'name': 'Novo Nome'},
  conflictAlgorithm: ConflictAlgorithm.replace,
);
```

Resultado:

O registro com `email = 'a@a.com'` terá o nome atualizado.

Gerenciamento de conflito

ConflictAlgorithm.rollback

Comportamento:

- Reverte **toda a transação**
- Retorna ao estado anterior ao início da transação
- Gera erro

Aplicação:

Em operações atômicas onde qualquer falha deve desfazer todas as mudanças.

```
await db.transaction((txn) async {
  await txn.insert(/* ... */);
  await txn.insert(
    'users',
    user.toMap(),
    conflictAlgorithm: ConflictAlgorithm.rollback,
  );
});
```

Gerenciamento de conflito

Tabela Comparativa

Algoritmo	Ação	Transação	Erro	Caso de Uso Típico
abort	Reverte comando	Mantém	Sim	Insert com tratamento de erro
fail	Mantém estado	Mantém	Sim	Operações não-atômicas
ignore	Não faz nada -	Mantém	Não	Dados não essenciais
replace	Substitui registro	Mantém	Não	Upsert (atualizar se existir)
rollback	Reverte tudo	Cancela	Sim	Transações complexas

Service Layer

Vantagens:

- **O que é?**
  Camada que coordena operações entre múltiplos DAOs e aplica regras de negócio.
- **Analogia:**
  "Maestro" que orquestra o fluxo de dados entre UI, modelos e persistência.
- Separar regras de negócio
- Facilita testes
- Reutilização

```
class AuthService {
  final UserDao _userDao = UserDao();

  Future<bool> register(User user) async {
    try {
      await _userDao.insertUser(user);
      return true;
    } catch (e) {
      return false;
    }
  }

  Future<User?> login(String email, String password) async {
    return await _userDao.getUser(email, password);
  }
}
```

Exercício

Criar crud alunos